

Unidad N° 4: Interacción con la base de datos

Introducción a Bases de Datos en Laravel

En esta nueva unidad, vamos a explorar cómo Laravel interactúa con las bases de datos, un componente esencial para el desarrollo de aplicaciones web dinámicas. Ya hemos cubierto los fundamentos del framework, y ahora es el momento de ver cómo podemos persistir y manipular datos de manera eficiente.

Conexión por defecto a SQLite

Cuando instalamos Laravel 12 hemos elegido por defecto trabajar con **SQLite**, una base de datos ligera basada en un archivo. Esta configuración nos permite comenzar a trabajar con la base de datos sin necesidad de instalar un servidor de base de datos adicional.

El archivo de configuración principal se encuentra en la raíz de tu proyecto, llamado `.env`. Este archivo de entorno contiene las variables que Laravel utiliza para la configuración de la base de datos.

```
DB_CONNECTION=sqlite
```

```
DB_DATABASE=database/database.sqlite # Opcional, si no se menciona el  
archivo se crea en esa ubicación
```

Como puedes ver, `DB_CONNECTION` está configurado como `sqlite`, y `DB_DATABASE` apunta a un archivo que se encuentra en la carpeta `database/`. Cuando seleccionamos SQLite al instalar el proyecto, Laravel ejecutó la migración y creó la base de datos con sus tablas en ese archivo.

Cambio a MySQL

Si ya tienes un servidor de MySQL instalado y prefieres usarlo, cambiar la configuración es muy sencillo. Solo necesitas modificar las variables de entorno en tu archivo `.env`.

Primero, debemos crear la base de datos en nuestro servidor de MySQL, ya sea mediante consola o la herramienta gráfica elegida. Luego debemos modificar las credenciales adicionales de conexión en el archivo `.env`:

```
DB_CONNECTION=mysql
```

```
DB_HOST=127.0.0.1
```

```
DB_PORT=3306
```

```
DB_DATABASE=nombre_de_tu_db
```

```
DB_USERNAME=tu_usuario
```

```
DB_PASSWORD=tu_contraseña
```

Aquí está el significado de cada una de estas variables:

- **DB_HOST:** La dirección IP o el nombre de host de tu servidor de MySQL.
- **DB_PORT:** El puerto en el que se ejecuta tu servidor de MySQL. El puerto por defecto es 3306.
- **DB_DATABASE:** El nombre de la base de datos que has creado.
- **DB_USERNAME:** El nombre de usuario que usas para acceder a MySQL.
- **DB_PASSWORD:** La contraseña de ese usuario.

Una vez que hayas modificado estos valores, guarda el archivo y estarás listo para interactuar con tu base de datos de MySQL usando Laravel.

Migraciones

Las **migraciones** son un mecanismo que nos permite definir la estructura de cada tabla de la base de datos dentro de la aplicación. Nos permiten definir el esquema de la base de datos (tablas, columnas, índices, etc.) utilizando código PHP. Esto tiene varias ventajas:

- **Colaboración:** Varios desarrolladores pueden trabajar en el mismo proyecto y mantener sus esquemas de base de datos sincronizados fácilmente.
- **Consistencia:** Asegura que la base de datos en diferentes entornos (desarrollo, producción) tenga la misma estructura.
- **Portabilidad:** Puedes migrar la base de datos sin preocuparte por el dialecto SQL específico de cada sistema (MySQL, PostgreSQL, etc.), ya que Laravel se encarga de traducirlo por ti.

Convención para la creación de migraciones

Para crear una nueva migración, utilizamos el siguiente comando en consola:

```
php artisan make:migration create_nombre_de_la_tabla_table
```

Laravel seguirá una convención para nombrar los archivos de migración: [timestamp]_create_[nombre_de_la_tabla]_table.php. Por ejemplo, 2024_08_12_100000_create_transacciones_table.php. El timestamp es crucial para que Laravel sepa en qué orden ejecutar las migraciones.

Los archivos de migración se encuentran en la carpeta database/migrations.

Métodos up() y down()

Cada clase de migración contiene dos métodos importantes:

- **up():** Este método se ejecuta cuando aplicamos la migración. Aquí definimos la lógica para crear o modificar las tablas de la base de datos.
- **down():** Este método se ejecuta cuando revertimos la migración. Su función es deshacer los cambios realizados en el método up(), por lo que generalmente se utiliza para eliminar tablas o columnas.

Para ejecutar una migración, se usa el comando:

```
php artisan migrate
```

Para revertir la última migración, se usa:

```
php artisan migrate:rollback
```

Para recrear completamente la base de datos desde cero, se usa:

```
php artisan migrate:fresh
```

NOTA: el comando fresh eliminará todos los datos ingresados en tus tablas, por lo cual es recomendable hacer un backup de los datos antes de ejecutarlo.

Ejemplo: Tabla de Transacciones

Ahora, vamos a crear una migración para una tabla de transacciones que contenga un ID, un monto, una descripción y la fecha de la transacción.

Ejecutamos el siguiente comando en consola:

```
php artisan make:migration create_transacciones_table
```

Eso creará un nuevo archivo llamado:

```
database/migrations/2025_08_12_100000_create_transacciones_table.php
```

Luego abrimos el archivo y modificamos el método up() para definir la estructura o “esquema” de la tabla transacciones y el método down() para eliminar la tabla:

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('transacciones', function (Blueprint $table) {
            $table->id();
            $table->string('descripcion');
```

```

        $table->decimal('monto', 8, 2);
        $table->date('fecha_transaccion');
        $table->timestamps();
    });
}

/**
 * Reverse the migrations.
 */
public function down(): void
{
    Schema::dropIfExists('transacciones');
}
};

```

En este ejemplo, la tabla transacciones tendrá las siguientes columnas:

- **id:** Un ID autoincremental de tipo BIGINT (big integer).
- **descripcion:** Una cadena de texto de tipo VARCHAR
- **monto:** Un número DECIMAL con un total de 8 dígitos, 2 de ellos para la parte decimal.
- **fecha_transaccion:** Una fecha de tipo DATE.
- **timestamps():** Crea automáticamente dos columnas: **created_at** y **updated_at**, que almacenan la fecha y hora de creación y la última actualización del registro, respectivamente.

Ejecutar la migración

Luego de creada la estructura de la tabla, es necesario migrarla para que impacte en la base de datos. Hasta que no la ejecutemos, nuestra base de datos no contará con la tabla.

Para ejecutar la migración ingresamos el comando:

```
php artisan migrate
```

Ahora podemos revisar nuestra base de datos y encontraremos la tabla.

Tipos de columnas

Estos son algunos de los tipos de columnas más comunes que podemos usar en nuestras migraciones para definir los campos de una tabla:

- **id()**: crea un campo INTEGER como clave primaria auto-incremental.
- **integer()**: crea un campo INTEGER común.
- **date()** o **dateTime()**: crea campos DATE o DATETIME.
- **decimal()** o **double()**: crea campos DECIMAL o DOUBLE.
- **timestamps()**: crea los campos especiales *created_at* y *updated_at* del tipo TIMESTAMP.
- **softDeletes()**: crea el campo especial *deleted_at* del tipo TIMESTAMP.
- **string()**: crea un campo VARCHAR.
- **text()**: crea un campo TEXT

Para más información: <https://laravel.com/docs/12.x/migrations#available-column-types>

Modificadores de columnas

Ahora mencionaremos los modificadores de columnas más usados, que nos permiten definir características adicionales para cada campo:

- **after('columna')**: permite agregar un campo después de una columna existente. Es el contrario al método **before('columna')**.
- **default(\$valor)**: define un valor por defecto para la columna.
- **nullable()**: permite que en el campo se ingresen valores NULL.
- **unsigned()**: define que los valores numéricos del campo no tengan signos.

Para más información: <https://laravel.com/docs/12.x/migrations#column-modifiers>

Índices

Para crear índices en nuestras migraciones podemos usar los siguientes métodos:

- **primary()**: crea un índice primario en el campo.
- **unique()**: crea un índice con valores únicos.
- **index()**: crea un índice común.
- **fullText()**: crea un índice de tipo full text.

Para más información: <https://laravel.com/docs/12.x/migrations#indexes>

Eloquent

Es el ORM (*Object Relational Mapper*) utilizado por Laravel para vincular las tablas de la base de datos con los datos de la aplicación. Cada modelo creado apunta a una tabla de nuestra base de datos y es el intermediario que usamos para realizar las consultas SQL.

Algunas de sus funcionalidades principales son:

- Mapeo de tablas mediante los Modelos.
- Interfaz de consultas SQL mediante la sintaxis de queries.
- Configuración de comportamiento mediante casts, mutadores, etc.

Modelos

Los modelos son clases especializadas que replican la estructura de una tabla de la base de datos. Por defecto, Eloquent lee automáticamente todos los campos de una tabla y los convierte en atributos de la clase modelo, lo cual significa que no tenemos que definir a mano esos atributos como código.

Además, podemos definir las relaciones de las tablas directamente en los modelos.

Todos los modelos de nuestra aplicación están en la carpeta `app/Models` y la convención dicta que debemos nombrar de forma singular al de la tabla que queremos vincular. Ejemplo: si mi tabla se llama “**transacciones**”, el modelo se llamará “**Transacción**”.

Crear un modelo

Para crear un modelo, usamos el siguiente comando:

```
php artisan make:model Transaccion
```

Esto creará una nueva clase en la carpeta `app/Models` con ese nombre. Su contenido será el siguiente:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Transaccion extends Model
{
    //
}
```

Configuración del modelo

Para que Laravel pueda vincular correctamente el modelo a la tabla, el modelo y la migración deben tener las siguientes propiedades:

1. El nombre del modelo será singular y el nombre de la tabla será plural. Como Laravel está escrito en inglés, simplemente agrega una “s” al final de la tabla para considerarlo plural. En este caso no se cumple con la norma porque nuestro modelo se llama Transaccion y la tabla definida en la migración es transacciones en lugar de transaccions.
2. La tabla cuenta con una clave primaria autoincremental de tipo entero que se llama id.
3. Por defecto contamos con los campos created_at y updated_at, que serán completados automáticamente por Laravel cuando se ejecuten las consultas SQL.

Si en cualquier modelo queremos sobrescribir esta configuración, porque uno o más de esas características no son correctas, podemos usar los siguientes atributos en nuestro modelo:

```
protected $table = 'mis_transacciones'; // Si mi tabla se llama de
forma diferente a la pluralización del modelo

protected $primaryKey = 'transaccion_id'; // Si mi clave primaria no
se llama id

public $incrementing = false; // Si mi clave primaria no es
autoincremental

protected $keyType = 'string'; // Si mi clave primaria no es entera

public $timestamps = false; // Si no vamos a utilizar los campos
created_at y updated_at

protected $guarded = []; // Deshabilitar protección de asignación
masiva
```

Para más información: <https://laravel.com/docs/12.x/eloquent#eloquent-model-conventions>

Consultas SQL

Todas las consultas SQL las haremos mediante los modelos, por ello es fundamental que hayamos configurado correctamente el modelo previamente para que se vincule correctamente con la tabla de la cual queremos obtener los datos.

Por lo general se ejecutan las consultas en los controladores de la aplicación.

Consultas SELECT

Para leer todos los datos de una tabla sin filtros usamos el método **all()**, el cual es equivalente a realizar una consulta del tipo “SELECT * FROM tabla”:

```
$transacciones = Transaccion::all();
```

Para filtrar esos datos mediante cierta condición usamos el método **where()** junto con el método **get()** al final del mismo:

```
$transacción = Transaccion::where("id", 2)->get();
```

Para obtener un dato específico según su id también contamos con el método **find()**, el cual devolverá siempre un solo registro:

```
Transaccion::find(1);
```

Como alternativa, podemos utilizar **where()** con la condición a buscar y al final **first()** para obtener solamente el primer registro en base al criterio:

```
Transaccion::where("id", 1)->first();
```

NOTA: si agregamos **first()** a cualquier consulta, siempre devolverá el primer registro en base al criterio definido previamente.

Consultas INSERT

La forma más fácil de crear un nuevo registro es utilizar el método **create()** junto a un arreglo asociativo con los datos deseados.

```
Transaccion::create([
    'descripcion' => 'Pago de telefonía móvil',
    'monto' => 20000,
    'fecha_transaccion' => date("Y-m-d"),
]);
```

NOTA: tanto el id primario, como los campos **created_at** y **updated_at** se rellenan automáticamente.

Opcionalmente, podemos crear un nuevo objeto de la clase modelo e ir asignando los valores a cada atributo/campo, para luego usar el método **save()** que guarda los valores en la BD:

```
$transaccion = new Transaccion();
$transaccion->descripcion = 'Pago de telefonía móvil';
$transaccion->monto = 20000;
$transaccion->fecha_transaccion = date("Y-m-d");
$transaccion->save();
```

Consulta UPDATE

Para actualizar un registro podemos utilizar el método **update()** – siempre teniendo en cuenta que debemos realizar un **where** previamente.

```
Transaccion::where('id', 1)
    ->update([
        'descripcion' => 'Compra de supermercado',
    ]);
```

Opcionalmente, podemos buscar primeramente el modelo con **find()**, luego actualizar el atributo/campo deseado y por último usar **save()** para guardarlo en la BD:


```
$transaccion = Transaccion::find(1);  
$transaccion->descripcion = 'Compra de supermercado';  
$transaccion->save();
```

Consulta DELETE

Se puede eliminar un registro mediante su clave primaria usando el método **destroy()**:

```
Transaccion::destroy(1);
```

O también utilizando un **where()**:

```
Transaccion::where('id', 1)->delete();
```

Borrado lógico

Si en lugar de eliminar un registro preferimos “ocultarlo” podemos utilizar la estrategia de borrado lógico, la cual implica definir un campo en la tabla que marque el campo como borrado para que no se muestre en las consultas, pero siga existiendo en la tabla. Esta estrategia suele implementarse por necesidades de mantener un historial. Por ejemplo: tenemos un cliente que ya no compra en nuestra aplicación, pero contamos con todo su historial previo que no deseamos borrar.

Laravel implementa el borrado lógico mediante la aplicación de **softDeletes()** en la migración y el modelo, lo cual crea un campo llamado **deleted_at** que indica la fecha en la cual se ha eliminado el registro.

Primero agregamos a la migración de la tabla la línea apropiada (por lo general se prefiero hacerlo al final):

```
Schema::table('transacciones', function (Blueprint $table) {  
    // Otros campos...  
    $table->softDeletes();  
});
```

Luego definimos en el modelo el Trait apropiado:

```
use Illuminate\Database\Eloquent\SoftDeletes;  
  
class Transaccion extends Model  
{  
    use SoftDeletes;  
    // Otras propiedades del modelo  
}
```

Ahora, cada vez que eliminemos un registro de esta tabla (ver sección anterior) este permanecerá en nuestra tabla con el campo **deleted_at** completo en la fecha y hora de eliminación y no se mostrará en las consultas SELECT regulares.

Operaciones con borrado lógico

Si en algún momento deseamos ver los registros “eliminados” mediante esta estrategia, podemos agregar el método **withTrashed()** a cualquiera de las consultas SELECT:

```
Transaccion::withTrashed()->all();
```

```
Transaccion::withTrashed()->where('id', 1)->get();
```

Si en algún momento deseamos restaurarlo, podemos usar el método **restore()**:

```
Transaccion::withTrashed()->where('id', 1)->restore();
```

Y si en algún momento deseamos eliminarlo definitivamente de nuestra tabla, podemos usar el método **forceDelete()**:

```
Transaccion::withTrashed()->where('id', 1)->forceDelete();
```